

Purin: A Biology-inspired Mechanism for Artificial Neural Networks

Zishu Liu¹, Chunbo Luo^{1*}, Christos Grecos²

¹University of Exeter

²University of Wisconsin - Parkside

zl538@exeter.ac.uk, C.Luo@exeter.ac.uk, grecoschristos@gmail.com

Abstract

Artificial neural networks (ANNs) usually represent neural transmission with fixed trainable weights during a training batch, which omits short-term changes in synaptic efficacy. In addition, the discrete time-step simulation requires additional temporal processing that many conventional ANN architectures do not use. To overcome these challenges, we propose Purin, a biology-inspired and ANN-compatible mechanism, that introduces synaptic efficacy modulation into conventional convolutional neural networks. Purin uses a time-interval-based abstraction for neural activities, which allows Purin to introduce short- and long-term synaptic efficacy changes without using discrete time-steps. Purin introduces a bounded factor to represent temporary synaptic efficacy changes, together with two weight matrices that represent input-side and output-side efficacy. The weight matrices are updated by back-propagation and interpreted as the long-term synaptic efficacy changes. Experimental results show that after removing the confounding factors in the AlexNet, VGG11, and GoogLeNet architectures, Purin improves the classification accuracies in all three models across the evaluated datasets.

1 Introduction

Artificial neural networks (ANNs) have been widely used in many areas, such as image classification (Krizhevsky, Sutskever, and Hinton 2012) and segmentation (Wang et al. 2020), natural language processing (Vaswani et al. 2017), and object detection (Kaur and Singh 2023). Although they have achieved promising performance in many areas, many ANNs are biologically implausible in various aspects such as the structure of neurons, the activation rule (action potential vs. activation functions), the learning rule, etc. Researchers try to narrow those gaps by proposing neural networks that introduce biological mechanisms. One of the most popular families is the spiking neural network (SNN) family, which introduces biologically plausible spiking neurons. However, they face difficulties in training the network because the spike generation functions are not differentiable. Thus, SNNs cannot directly use the backpropagation (BP) algorithm (Tavanaei et al. 2019).

In this paper, we propose a biology-inspired mechanism called Purin to reduce selected gaps between ANN and biological neurons while making it compatible with represen-

tative ANN architectures and the BP learning rule. Firstly, we develop a biology-inspired abstraction for biological neurons, which introduces neurons with two split weight matrices that reflect the efficacy of the input-side and output-side efficacy terms, respectively. Secondly, we introduce the time interval in Purin in a more conceptual way, which treats a still image as information presented in a short time interval. It allows Purin to introduce short- and long-term weight dynamics without extra transformation steps (e.g., spike encoding). Thirdly, inspired by the short- and long-term plasticity in biology, we introduce a short-term factor g_{stp} in Purin that introduces activation-dependent changes to the network. The short-term factor causes temporary weight changes within a batch, which also affects the BP-based long-term update in the weight matrices.

2 Related work

ANNs are inspired by biological neurons, but focus more on the principles such as function optimization than biology (Marblestone, Wayne, and Kording 2016a). Despite the fact that researchers have tried to explain or approximate the BP algorithm in various ways (Marblestone, Wayne, and Kording 2016b; Whittington and Bogacz 2019), the differences between biological neurons and the ANN neurons, such as spike-based versus continuous-vector outputs and distinct learning rules, eventually motivate the development of the SNN family.

SNNs introduce a more biologically plausible neuron model that emits spikes when the membrane potential reaches the firing threshold. The synapses (or weights) in SNNs are updated based on biological principles and spike-timing dependent plasticity (Markram, Gerstner, and Sjöström 2011) rather than the BP algorithm. However, training deep SNNs based on discrete spikes is challenging because it cannot use derivative-based optimization methods like ANNs (Taherkhani et al. 2020). To overcome this difficulty, researchers proposed the surrogate gradient for training SNNs (Zenke and Ganguli 2018; Bellec et al. 2018). Another method to address the training difficulty in deep SNN is ANN-to-SNN conversion, which converts pre-trained ANN models to SNN models to make SNN models compatible with existing ANN models and reduce the resources for training (Hu et al. 2024). In addition to the training difficulty, SNNs require encoding images into spikes, which is less

*Corresponding author.

common for many existing image classification datasets. As a result, SNNs usually need additional preprocessing steps in comparison to ANNs. Although researchers have tried the idea of combining a CNN feature extractor and an SNN to make final decisions (Xu et al. 2022), additional spike encoding is still necessary in this model.

The short-term plasticity of synapses has been explored by many researchers. In (Abbott et al. 1997), the authors suggested that the short-term depression of synapses provides a dynamic gain-control mechanism. The short-term depression can be described by a multiplicative factor in the model mentioned in that paper. A survey (Henning 2013) summarizes theoretical models of short-term synaptic plasticity. These models focus on different points that cause the short-term synaptic plasticity, and many of them suggest that this mechanism can be written as a multiplicative factor in mathematical equations. However, the models mentioned above are usually based on spiking or discrete time steps, which is incompatible with the requirement of continuous inputs or outputs in ANNs.

In deep learning, attention and gating methods are usually used to learn sample-dependent features. The Squeeze-and-Excitation (SE) block proposed in (Hu, Shen, and Sun 2018) performs channel-wise feature recalibration by modeling interdependencies between the convolution channels. However, the SE block neglects important position information in the attention maps. Thus, coordinate attention (Hou, Zhou, and Feng 2021) is proposed to capture both channel-wise information and long-range positional information. In terms of gating methods, the long short-term memory (Hochreiter and Schmidhuber 1997) introduced three gates (the input gate, the forget gate and the output gate) to enable the network to keep important long-term memory information. The gMLP (Liu et al. 2021) introduced a spatial gating unit, which used a linear projection matrix independent of the inputs of unit, to allow cross-token interactions. Despite this technique being successful, many attention and gating methods make sustainable changes to the trainable parameter, which is different from the short-term plasticity in biology (Regehr 2012).

3 Method

In this section, we describe the proposed Purin mechanism in detail. In general, an ANN neuron with Purin mechanism is formulated as Eq (1) where X is its input, W_{in} and W_{out} are two weight matrices that represent input-side and output-side efficacy of the neuron and $f(\cdot)$ is the signed magnitude function, which is a signed activity measure over a short time interval. The symbol $\odot$ denotes Hadamard product. Variable g_{stp} is a factor that reflects the temporary efficacy changes caused by the short-term plasticity. In fully connected layers, g_{stp} is a vector that has the same shape as neuron’s output vector. In 2D convolution layers, g_{stp} is a vector with C elements that matches the number of convolution kernels in that layer, which performs channel-wise multiplication in convolution layers.

$$Y = g_{stp} \odot W_{out} \odot f(W_{in}X) \quad (1)$$

In the following sections, we introduce our motivation, the abstraction of biological neurons, the short-term plasticity,

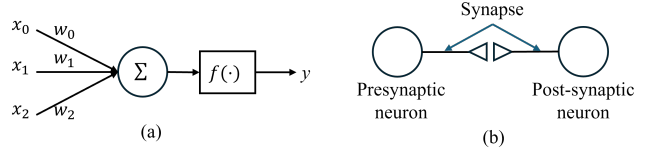


Figure 1: A schematic diagram of neurons in ANNs and biology. (a) neuron in ANN. (b) neuron in biology.

and justify why there is no bias vector and dropout in our mechanism.

3.1 Motivation

The motivation of Purin is to narrow the selected gaps between ANNs and biological neurons, while preserving compatibility with continuous activations and the BP algorithm. Thus, Purin is not intended to be a biophysically faithful model of synaptic plasticity. Instead, Purin is designed based on two biological observations: the synaptic efficacy changes temporally due to the short-term plasticity, and the signal transmission between two neurons depends on the efficacy of pre- and post-synaptic components of the synaptic connection.

As shown in Fig. 1, neurons in ANNs usually do not have output weights after generating the output y . By contrast, a biological chemical synapse involves both presynaptic and postsynaptic components that contribute to transmission efficacy. This structural difference further leads to differences in transmitting the neuron’s outputs (see Fig. 2). ANN neurons simplify the procedure of transmitting the action potential in biological neurons. However, the synaptic efficacy of presynaptic components plays a role in information transmission between neurons, and the synapse plasticity mechanism also works on the presynaptic component. Thus, simplifying presynaptic and postsynaptic components into one ignores the enhancement or attenuation caused by the previous neuron when transmitting the information. In addition, the trainable weights in ANNs are usually fixed during a training batch, while synapses in biological neurons show temporal dynamic changes due to the short-term plasticity mechanism, which can eventually lead to long-term changes in synaptic efficacy.

In Purin, we introduce a simplified form of synaptic efficacy modulation into CNN architectures without requiring spike generation, spike encoding, or discrete temporal simulation. The following subsections describe how output-side efficacy, abstract time intervals, and short-term modulation are formulated in Purin.

3.2 Biology-inspired Abstraction of Neuron

In this section, we propose a new abstraction for biological neurons that is compatible with the BP algorithm. We use a neuron shown in Fig. 3 as an example to illustrate how we abstract the biological neuron into the proposed mechanism.

Output Efficacy We start by observing that signal transmission across a chemical synapse depends on the neurotransmitter, which is affected by the efficacy of the synaptic

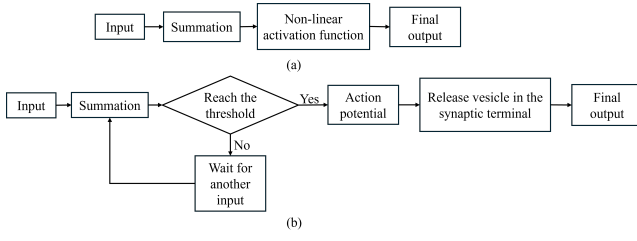


Figure 2: A flowchart describing how an ANN’s neuron and biological neuron process their inputs. We focus on the biological neuron using chemical synapses. (a) ANN’s neuron’s procedure. (b) biological neuron’s procedure.

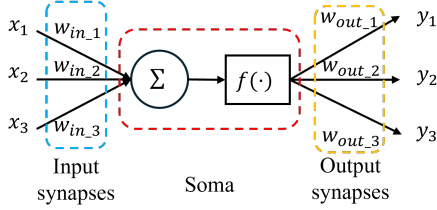


Figure 3: An ANN neuron with Purin mechanism. This neuron receives 3 inputs (x_1, x_2, x_3) from the previous layer and generates 3 outputs (y_1, y_2, y_3) to 3 neurons in the next layer.

terminal. The proposed mechanism abstracts it as the output-side efficacy of ANN neurons, and introduces an efficacy factor W_{out} to each neuron after the activation function. The proposed scheme does not take into account the decay of the signal during transmission because it is difficult to define the "distance" between neurons in ANNs. The equation of the classic ANN neuron is now rewritten as Eq (2), where $f(\cdot)$ is the activation function. As W_{out} represents the efficiency of transmitting the signal in the presynaptic neuron, it should preserve the sign of the result of $f(\cdot)$. Therefore, in Purin, w_{out_i} is constrained to a non-negative number so that it scales the result of $f(\cdot)$ without reversing its sign.

$$y_j = w_{out_i} \times f\left(\sum_{i=1}^3 w_{in_i} x_i\right) \quad (2)$$

From Time Step to Time Interval As most ANN models for image classification do not use temporal inputs and outputs, Purin requires an abstraction that introduces short-term dynamics without constructing discrete time steps. However, a single still image cannot form a temporal image sequence from the view of the ANN setting. On the other hand, research on rapid serial visual presentation suggests that biological visual recognition occurs over a short presentation interval rather than at an instantaneous moment (Mohsenzadeh et al. 2018; Maguire and Howe 2016). This conclusion provides the idea that an image can be abstracted as information presented in a short time interval.

The abstraction of input images is also required to introduce long-term and short-term plasticity to ANN models. In ANN architectures, the trainable weights remain unchanged

within a training batch and are updated after one training batch is completed. This batch-based update can be treated as long-term synaptic efficacy changes in neural activities. But biological synapses can exhibit short-term efficacy changes before long-term synaptic changes are established. Therefore, if we want to introduce short-term synapse plasticity to ANNs while keeping BP-based long-term optimization, the feedforward process should be described using an interval-based abstraction rather than an instantaneous event.

Based on this view, we define the input images as the information presented in a short time interval. The time interval does not mean a specific physical duration. It represents a period in which long-term plasticity does not occur in the long-term weight matrices W_{in} and W_{out} , while the short-term factor g_{stp} changes according to the activity generated by the current input. After a training batch is completed, W_{in} and W_{out} update their value based on the gradient accumulated in the batch using the BP algorithm like ANNs, and g_{stp} is reset to its default value. This abstraction allows Purin to introduce short-term and long-term plasticity mechanisms without spike encoding or simulating discrete time steps.

To meet the time interval abstraction, the activation function $f(\cdot)$ in ANN should not only represent the neural activation at a specific moment. In Purin, we interpret $f(\cdot)$ as a signed activity measure rather than as the absolute firing rate of a biological neuron. We name $f(\cdot)$ as signed magnitude function. Suppose a neuron has a baseline firing rate M during a time interval. If its firing rate is higher than M in a time interval, this neuron shows excitatory-like activity relative to its baseline. If its firing rate is lower than M in a time interval, this neuron shows inhibitory-like activity. Since Purin does not model the firing rate, rather it abstracts the activity relative to the baseline, we can centre the baseline value M at zero. If the value of $f(\cdot)$ is greater than 0, it indicates that the neuron’s activity is above the baseline. If the value of $f(\cdot)$ is smaller than 0, it indicates that the neuron’s activity is below the baseline. The absolute value of $f(\cdot)$ reflects the magnitude of the deviation. According to this interpretation, a neuron can have a negative or positive output in the proposed abstraction, which shows whether its activity is below or above its baseline.

Since $f(\cdot)$ abstracts the neural activity in a short time interval, the weight matrices W_{in} , W_{out} and the short-term factor g_{stp} represent the efficacy during information transmission. In this light, the output of a Purin neuron can also be interpreted as transmitted activity over a short interval. Thus, we can apply this abstraction from the first layer to other deeper layers in the neural network.

3.3 Short-term Plasticity

The short-term plasticity is an important mechanism that refers to temporary changes in synaptic efficacy over short time scales. These changes can arise from presynaptic and/or postsynaptic mechanisms. For a biological faithful model, we may need to rewrite Eq (2) into Eq (3) shown below, where Δw_{in_i} and Δw_{out_i} are the efficacy changes caused

by short-term plasticity.

$$y_j = (w_{out_i} + \Delta w_{out_i}) \times f\left(\sum_{i=1}^3 (w_{in_i} + \Delta w_{in_i}) x_i\right) \quad (3)$$

However, this equation introduces a challenging problem that requires distinguishing where the short-term plasticity occurs and quantifying its strength. This issue is difficult because error signals in ANNs, such as gradients, are rarely applied to the formulation with split weights. Therefore, it is difficult to distinguish which component of the synaptic connection is affected by the error signals. To address this issue, we introduce a factor g_{stp} to represent the efficacy changes caused by short-term plasticity, as shown in Eq (4).

$$y_j = g_{stp_i} \times w_{out_i} \times f\left(\sum_{i=1}^3 w_{in_i} x_i\right) \quad (4)$$

The factor g_{stp} will change over time and gradually reset to its default value 1.0, which means the efficacy is 100%. This will happen after the weights w_{out_i} and w_{in_i} are updated because updating the weights implies the long-term plasticity has occurred. Therefore, resetting g_{stp} to the default value at that point will ensure that the new synapses will not be affected by their status before updating.

Based on the time interval abstraction in 3.2, g_{stp} updates in the feedforward procedure for every input image. Eq (5) shows how g_{stp} updates, where λ is the learning rate.

$$g_{stp_new} = g_{stp_old} + \lambda f\left(\sum_{i=1}^3 w_{in_i} x_i\right) \quad (5)$$

To avoid g_{stp} becoming infinitely large or small, its value should be bounded. In this paper, g_{stp} is limited from 0.8 to 1.2 to provide a mild modulation factor to the output.

As the efficacy changes caused by short-term plasticity automatically vanish over time, g_{stp} recovers to its default value. Eq (6) shows how g_{stp} recovers to the default value over time, where d is the default value of g_{stp} and b is the batch size. Since the long-term plasticity occurs with BP-based weight update, which triggers when a batch of samples is completed, we can define the long time period in our paper as the number of images in one batch.

$$g'_{stp} = g_{stp} + (d - g_{stp}) \times (1 - \exp(-1/b)) \quad (6)$$

For 2D convolution layers, we average the sum of the function outputs (i.e., the sum of $f(\cdot)$ in a channel) to update g_{stp} in a channel. For a 2D convolution neuron with N elements in its output matrix, Eq (7) shows how to update g_{stp} in the feedforward procedure, where s denotes the sum of the function outputs and λ is the learning rate.

$$g_{stp_new} = g_{stp_old} + \lambda s/N \quad (7)$$

3.4 Recovery Analysis of Short-term Factor

The dynamic short-term factor g_{stp} affects the outputs of neurons in the Purin mechanism, and it is necessary to show its stability. Since we already set a boundary to g_{stp} , it does not lead the output y_i to infinity. Thus, in this section, we

discuss whether it recovers to the default value d . We assume that the inputs of the neuron are 0 when analyzing the recovery of g_{stp} , which makes it clear whether the recovery step leads g_{stp} to d .

Starting from Eq (6) and denoting $1 - \exp(-1/b)$ as ρ , we need to measure the distance between g'_{stp} and the default value d . Thus, we subtract d from both sides of Eq (6) and rewrite it as Eq (8) by grouping g_{stp} and d on the right side.

$$g'_{stp} - d = (1 - \rho)g_{stp} + (\rho - 1)d \quad (8)$$

It is obvious that $\rho - 1 = -(1 - \rho)$, so Eq (8) can be rewritten as Eq (9).

$$g'_{stp} - d = (1 - \rho)(g_{stp} - d) \quad (9)$$

Since $\rho = 1 - \exp(-1/b)$, where b is the batch size (thus positive), $1 - \rho = \exp(-1/b)$ is in the range of $(0, 1)$. Eventually, we can rewrite Eq (9) to Eq (10) that shows that the distance between g'_{stp} and the default value d keeps shrinking after every recovery step.

$$g'_{stp} - d = (g_{stp} - d)\exp(-1/b) \quad (10)$$

3.5 Weight Update

The weight matrices W_{in} and W_{out} can be trained by gradient descent as ANNs. Based on Eq (4), we present how to update w_{out_i} and w_{in_i} in Eq (11) to (14), where η is the learning rate.

$$\Delta w_{out_i} = \frac{\partial L}{\partial y_j} \times g_{stp_i} \times f\left(\sum_{i=1}^3 w_{in_i} x_i\right) \quad (11)$$

$$w_{out_i} = w_{out_i} - \eta \Delta w_{out_i} \quad (12)$$

$$\Delta w_{in_i} = \frac{\partial L}{\partial y_j} \times g_{stp_i} \times w_{out_i} \times f'\left(\sum_{i=1}^3 w_{in_i} x_i\right) \times x_i \quad (13)$$

$$w_{in_i} = w_{in_i} - \eta \Delta w_{in_i} \quad (14)$$

In 2D convolution layers, the convolution kernel is shared within an output channel. As a result, there is one output weight for each kernel, and the equation for calculating ΔW_{out} is rewritten as Eq (15). In this equation, Y is the output feature map for this channel, $\frac{\partial L}{\partial Y}$ is the backpropagation error signal, N is the number of elements in Y , and $*$ denotes channel-wise multiplication.

$$\Delta W_{out} = g_{stp} * \sum_{i=1}^N (f(W_{in} X) \odot \frac{\partial L}{\partial Y}) \quad (15)$$

The calculation of elements in ΔW_{in} is very similar to Eq (13), the only difference is that g_{stp_i} is shared within a channel.

3.6 The Bias Vector and Dropout Mechanism

In ANNs, the bias vector $\mathbf{b}$ inside the activation function provides flexibility to the neuron. Through biological abstraction, the bias vector plays a role that is similar to the firing threshold. However, this potential explanation of the bias vector works when simulating neuron's activities at a specific moment, which is incompatible with the time interval abstraction in the proposed mechanism. Although we

attempted to identify a biologically plausible meaning for the bias vector under our time period condition, we have not yet found a satisfactory answer. Therefore, we exclude the bias vector from our mechanism.

The dropout mechanism (Srivastava et al. 2014) is a commonly used mechanism to prevent ANNs from overfitting by randomly setting the output of a neuron to 0 according to a probability p . Although this mechanism can be loosely related to unreliable neural transmission, it scales the weight during testing, which is a mathematical approximation for deterministic inference results rather than a biological mechanism. More importantly, g_{stp} updates its value based on the inputs of the neuron. If the dropout mechanism is applied, the sum of the inputs will become stochastic due to the random mask, which leads g_{stp} to be updated based on corrupted activity rather than the true activity from presynaptic neurons. Therefore, we do not preserve the dropout mechanism in Purin.

4 Experiments

In this section, we introduce our method to three benchmark deep learning models, AlexNet (Krizhevsky, Sutskever, and Hinton 2012), VGG11 (Simonyan and Zisserman 2014), and GoogLeNet (Szegedy et al. 2015) to evaluate its performance. We replace the vanilla weight matrices with our split weight matrices mentioned in Section 3.

4.1 Datasets and Experimental Settings

We evaluated our mechanism on four widely used datasets: CIFAR-100 (Krizhevsky and Hinton 2009), AID (Xia et al. 2017), UCM (Yang and Newsam 2010), and 102 Category Flower Dataset (Nilsback and Zisserman 2008). The relevant details of them are briefly listed below.

1. CIFAR-100: this dataset contains 50000 training samples and 10000 test samples from 100 categories, with a size of 32×32 pixels in each image.
2. AID: this dataset contains 10000 aerial images from 30 categories, with a size of 600×600 pixels in each image.
3. UCM: this dataset contains 2100 aerial images from 21 categories, with a size of 256×256 pixels in each image.
4. 102 Category Flower Dataset: This dataset contains 8189 flower images with various image sizes. Each class consists of between 40 and 258 images. In this paper, we denote this dataset as Oxford102.

We resized the images in all four datasets to 224×224 pixels to match the input shape of AlexNet and GoogLeNet architectures. We resized the images in the AID, UCM, and Oxford102 datasets to 224×224 pixels for VGG11 to avoid the out-of-memory problem on our device. We randomly selected 10% of the training samples in the CIFAR-100 dataset as the validation set. The AID and UCM datasets are divided into training, validation, and test sets with proportions of 60%, 20%, and 20%, respectively. We used the official split of the Oxford102 dataset which contains 1020 images in the training and validation set, respectively, and 6149 images in the test set.

All experiments were run on a computer with a NVIDIA RTX A5000 graphic card and 24 GB of memory. The coding environment for vanilla models included Ubuntu 24.04, Python 3.11, and PyTorch 2.4.0 with CUDA 11.8. We implement our method on the same device in C/C++ and CUDA which was compiled with a NVCC compiler v11.7. The batch size was set to 100. The AdamW optimizer with a learning rate of 0.0001 and a weight decay of 0.0005 were used for all models in our experiments. We used Leaky ReLU ($\alpha = 0.01$) as the signed magnitude function $f(\cdot)$ for the models that introduced our mechanism. The learning rate λ for g_{stp} was set to 0.0001. Input images were scaled to the range $[-1.0, 1.0]$ for models with Purin mechanism. The elements in the output synapse matrices W_{out} were initialized from a uniform distribution $U(0.9, 1.1)$. The g_{stp} and W_{out} were disabled in the last fully connected layer, which meant that their values were always 1, to ensure stable outcomes. In Purin-based models, we did not use batch normalization. This design focuses on the effect of split weight and short-term factor in Purin without introducing additional normalization mechanisms. The short-term factor g_{stp} is reset to its default value (1.0) when every epoch is completed and before the training, validation, and testing stages begin, to avoid potential information leakage.

Although the pre-trained weights of the aforementioned models could be loaded into models with our mechanism, they are obtained without the split synaptic mechanism. As a result, they will mismatch the distribution of input features under our mechanism due to the changes of the activation function and neuron’s input. Thus, we decided to train all models from scratch to make fair comparisons.

4.2 Comparisons with Benchmark Models

In this section, we compare the overall classification accuracy (OA) of vanilla models and models with our mechanism. The experimental results, which are the average accuracy over five independent experiments, are shown in Table 1.

Model	UCM	AID	CIFAR-100	Oxford102
AlexNet	64.5 $\pm$ 0.9	61.3 $\pm$ 1.8	48.9 $\pm$ 0.5	17.4 $\pm$ 2.3
AlexNet+P	72.2 $\pm$ 1.6	59.4 $\pm$ 1.0	45.9 $\pm$ 0.4	26.8 $\pm$ 0.9
VGG11	51.9 $\pm$ 2.4	55.4 $\pm$ 1.5	32.4 $\pm$ 0.4	7.4 $\pm$ 0.9
VGG11+P	68.2 $\pm$ 2.6	57.8 $\pm$ 1.4	39.8 $\pm$ 0.8	23.8 $\pm$ 0.7
GoogLeNet	76.7 $\pm$ 2.5	63.3 $\pm$ 3.9	55.0 $\pm$ 0.6	33.2 $\pm$ 3.2
GoogLeNet+P	74.0 $\pm$ 1.7	64.2 $\pm$ 2.0	52.8 $\pm$ 0.5	27.6 $\pm$ 2.3

Table 1: OA (%) on different datasets. The average accuracy and the standard deviation are given. "P" means Purin mechanism.

The experimental results show that our mechanism improves the OA of VGG11 on all four datasets. Our mechanism significantly improves OA of VGG11 on the Oxford102 dataset by approximately 3 times. It also improves OA of VGG11 by about 17% on the UCM dataset and 7% on the CIFAR-100 dataset. When incorporated with AlexNet, our mechanism improves the OA on the UCM and Oxford102 datasets by about 8% and 9%, respectively. However, on the

AID and CIFAR-100 datasets, models with our mechanism show about 2% to 3% drop in OA. Results on the GoogLeNet architecture show that our mechanism cannot help improve OA on the UCM, CIFAR-100, and Oxford102 datasets, and only slightly improves the OA on the AID dataset by about 0.9%.

The experimental results shown above cannot be considered as fair comparisons because the Purin and vanilla models have differences in input pre-processing, activation function, and the use of the dropout and batch normalization mechanism (see Table 2). Therefore, it is difficult to identify whether the improvement and decrease in accuracy are caused by the proposed mechanism or not.

Confounding factor	Models with Purin	Vanilla models
Input rescaling	[-1.0, 1.0]	[0.0, 1.0]
Activation function	Leaky ReLU	ReLU
Use dropout	No	Yes
Use batch normalization	No	Yes

Table 2: Confounding factors between vanilla models and models with Purin.

To estimate the effect of confounding factors, we evaluated matched benchmark models that use the same input rescaling, activation function, and dropout usage as models with Purin. The experimental results over five independent experiments are shown in Table 3.

Model	UCM	AID	CIFAR-100	Oxford102
AlexNet	61.3 $\pm$ 0.6	54.1 $\pm$ 1.1	42.1 $\pm$ 0.6	16.6 $\pm$ 2.2
AlexNet+P	72.2 $\pm$ 1.6	59.4 $\pm$ 1.0	45.9 $\pm$ 0.4	26.8 $\pm$ 0.9
VGG11	51.3 $\pm$ 3.4	50.2 $\pm$ 1.3	33.3 $\pm$ 1.1	10.5 $\pm$ 0.7
VGG11+P	68.2 $\pm$ 2.6	57.8 $\pm$ 1.4	39.8 $\pm$ 0.8	23.8 $\pm$ 0.7
GoogLeNet	49.9 $\pm$ 3.3	53.8 $\pm$ 1.5	39.2 $\pm$ 0.9	14.4 $\pm$ 2.1
GoogLeNet+P	74.0 $\pm$ 1.7	64.2 $\pm$ 2.0	52.8 $\pm$ 0.5	27.6 $\pm$ 2.3

Table 3: Comparisons with matched benchmark models in OA (%). The average accuracy and the standard deviation are given. "P" means Purin mechanism.

Compared with matched benchmark models, Purin can improve OA for the matched models on all four datasets. For the Alexnet and VGG11 architectures, Purin improves the OA on the Oxford102 and UCM datasets by approximately 10% for both models. On the AID and CIFAR-100 datasets, Purin improves the OA of the two models by approximately 3% to 7%. Without the dropout and batch normalization mechanisms, the OAs of the matched GoogLeNet architecture on four datasets significantly drop by 10% to 26%. With the help of the proposed mechanism, the accuracies of the Purin+GoogLeNet model reach a much closer value to the vanilla GoogLeNet (see Table 1), especially on the UCM and CIFAR-100 datasets where Purin helps recover the OA by about 24% and 13%, respectively. Although the OA of the Purin+GoogLeNet model is still about 6% lower than the vanilla GoogLeNet on the Oxford102 dataset, it recovers

about 13% of OA compared to the matched model on the same dataset.

4.3 Ablation Study

In this section, we explore the effects of different parts in our mechanism. We list the experimental results over five independent experiments in Table 5.

Variant	Split weight	g_{stp}
A	-	-
B	✓	-
C	✓	✓

Table 4: Ablation variants

Model	Variant	UCM	AID	CIFAR-100	Oxford102
AlexNet	A	61.3 $\pm$ 0.6	54.1 $\pm$ 1.1	42.1 $\pm$ 0.6	16.6 $\pm$ 2.2
	B	72.4 $\pm$ 0.9	58.7 $\pm$ 1.2	44.5 $\pm$ 0.4	26.6 $\pm$ 1.2
	C	72.2 $\pm$ 1.6	59.4 $\pm$ 1.0	45.9 $\pm$ 0.4	26.8 $\pm$ 0.9
VGG11	A	51.3 $\pm$ 3.4	50.2 $\pm$ 1.3	33.3 $\pm$ 1.1	10.5 $\pm$ 0.7
	B	65.4 $\pm$ 2.5	56.7 $\pm$ 0.8	38.0 $\pm$ 0.9	23.5 $\pm$ 0.6
	C	68.2 $\pm$ 2.6	57.8 $\pm$ 1.4	39.8 $\pm$ 0.8	23.8 $\pm$ 0.7
GoogLeNet	A	49.9 $\pm$ 3.3	53.8 $\pm$ 1.5	39.2 $\pm$ 0.9	14.4 $\pm$ 2.1
	B	74.7 $\pm$ 1.4	61.7 $\pm$ 0.9	47.7 $\pm$ 0.7	27.0 $\pm$ 1.9
	C	74.0 $\pm$ 1.7	64.2 $\pm$ 2.0	52.8 $\pm$ 0.5	27.6 $\pm$ 2.3

Table 5: OA (%) of ablation variants on four datasets.

The experimental results indicate that the split weight mechanism contributes the most to the improvement of OA. Compared to variant A, in most cases, the classification accuracies of variant B for all three architectures are improved by at least 4% on four datasets. The short-term factor g_{stp} can further improve accuracy of variant B by approximately 0.2% to 5% in most cases, which is a smaller magnitude of improvement compared to that brought by the split weight mechanism.

In terms of architecture, the VGG11 architecture always benefits from the Purin mechanism on all four datasets, whereas the AlexNet and GoogLeNet architectures benefit from the proposed mechanism on all datasets except the UCM one. However, compared with variant B of the two architectures, the accuracy only decreases by 0.2% for AlexNet and 0.7% for GoogLeNet. These minor differences cannot provide strong evidence that the full Purin mechanism hurts the performance of the two architectures. In conclusion, compared to the matched models and models that use the split weight mechanism only, the combination of split weights and g_{stp} can improve the OA of the architectures listed in this paper.

4.4 Short-term Factor Variants

In this section, we compare four variants of g_{stp} , which are indicated as variants A to D in Table 6, with the one proposed in Purin (variant E) on the CIFAR-100 and UCM datasets. The variant A means g_{stp} is generated from $U(0.8, 1.2)$ after one

sample is completed. We used the VGG11 and GoogLeNet architectures for the experiments. The experimental results over five independent experiments are listed in Table 7.

Variant	exponential recovery	range
A	-	$g_{stp} \sim U(0.8, 1.2)$
B	-	(0.8, 1.2)
C	✓	(0, $+\infty$)
D	-	(0, $+\infty$)
E	✓	(0.8, 1.2)

Table 6: Variants of g_{stp}

Architecture	Variant	CIFAR-100	UCM
VGG11	A	37.6 ± 0.4	62.1 ± 1.6
	B	38.6 ± 0.7	65.7 ± 2.3
	C	38.9 ± 0.4	66.0 ± 1.2
	D	38.6 ± 1.0	67.2 ± 1.7
	E	39.8 ± 0.8	68.2 ± 2.6
GoogLeNet	A	30.5 ± 0.3	58.19 ± 1.7
	B	52.8 ± 0.9	73.6 ± 4.6
	C	52.0 ± 0.9	73.3 ± 2.2
	D	51.9 ± 1.0	73.5 ± 1.0
	E	52.8 ± 0.5	74.0 ± 1.7

Table 7: OA (%) for architectures with different g_{stp} variants

According to the results in Table 7, variant A always achieves the lowest OA on the two datasets for both VGG11 and GoogLeNet architectures. Although the OA gap between variant A and other variants is only 2-6% for the VGG11 architecture, it becomes much larger for GoogLeNet where variant A performs approximately 18% worse than other variants. This result suggests that g_{stp} is not equivalent to random noise and provides better results. The OA values of the other four variants are similar, with differences of approximately 1%, but variant E achieves the highest accuracy. In addition, the performances of variants B, C, and D are mixed in these experiments, and it is difficult to tell under what conditions these variants are better choices. However, the mechanisms applied to g_{stp} in Purin help models achieve consistent performance across different architectures and datasets. Therefore, exponential recovery and limited value range of g_{stp} provide more stable and better performance in classification accuracy when combined with each other.

4.5 Comparisons with the SE Block

In (Hu, Shen, and Sun 2018), authors developed SE block that performs channel-wise multiplication on the output of the Inception module. This looks similar to the $g_{stp} \odot W_{out}$ operation in convolution layers in the GoogLeNet+Purin model. Thus, we evaluated the performance of the matched GoogLeNet architecture with SE blocks inserted after the convolution layer or inserted after the inception module as the original paper (Hu, Shen, and Sun 2018). Since the GoogLeNet has only one fully connected layer as the classifier head, whose W_{out} and g_{stp} are disabled when incor-

porating with Purin mechanism, we can focus on the effect of the SE blocks and the $g_{stp} \odot W_{out}$ operation on the classification accuracy. The average OA and standard deviation over five independent experiments are shown in Table 8. We also show the results of the matched GoogLeNet.

Model	CIFAR-100	UCM
GoogLeNet+conv SE block	38.4 ± 1.1	56.4 ± 0.9
GoogLeNet+SE block	35.9 ± 1.9	57.7 ± 4.3
GoogLeNet (matched)	39.2 ± 0.9	49.9 ± 3.3
GoogLeNet+Purin	52.8 ± 0.5	74.0 ± 1.7

Table 8: OA (%) on CIFAR-100 and UCM for different combinations of matched GoogLeNet

According to the results shown above, all three matched GoogLeNet models (with or without the SE block) do not outperform the combinations of GoogLeNet+Purin. Their OA are approximately 13% to 25% lower than that of the GoogLeNet+Purin variant on two datasets. The form of inserting SE blocks after all convolution layers is similar to how Purin applies g_{stp} and W_{out} , but the OA of that combination is about 17% to 20% lower than that of GoogLeNet with full Purin mechanism on both datasets. These results show that the $g_{stp} \odot W_{out}$ operation does not behave as a weaker SE-style recalibration module.

5 Conclusions and Future Work

We propose a biology-inspired mechanism, Purin, that introduces the output-side efficacy and the short-term plasticity to deep learning architectures for image classification tasks. Purin does not require transforming images to spike encoding like SNNs, and is compatible with the gradient descent and BP algorithm in ANNs due to its time interval based abstraction. The experimental results show that, compared with vanilla baseline models, models with Purin mechanism do not consistently improve all architectures. After removing confounding factors from vanilla architectures, models with the Purin mechanism achieved higher classification accuracy on all four datasets for all matched models.

The ablation experiments indicate that, in most cases, the full Purin mechanism can further improve the classification accuracy compared to models that only use the split weight mechanism. The exponential recovery and value range limitation for g_{stp} can achieve higher OA for the Purin mechanism. Comparisons with the SE blocks suggest that the $g_{stp} \odot W_{out}$ operation does not behave as a weaker SE-style recalibration module.

This paper does not evaluate Purin with residual architectures such as ResNet18 (He et al. 2016) due to differences in neural structure. The extension of Purin to the residual connection mechanism is a possible research topic in future works. We provide a simple experiment as an example of a possible way to incorporate Purin into the residual connection in the supplementary material.

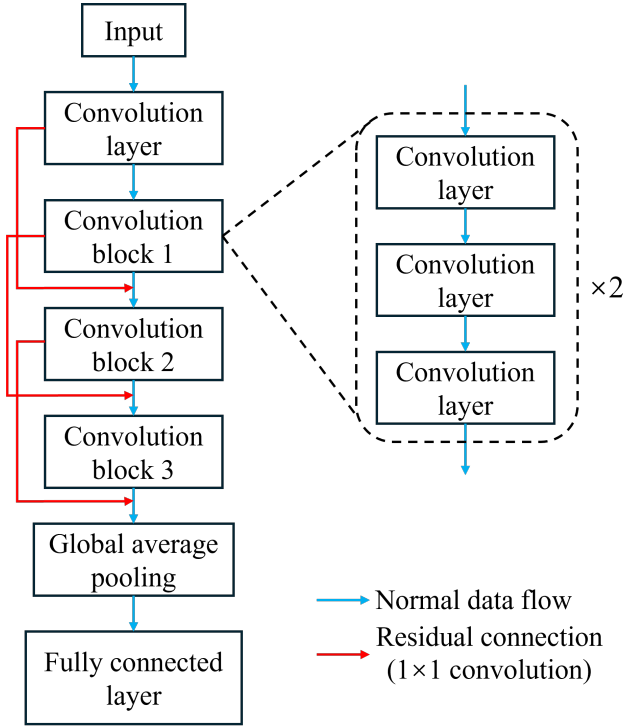


Figure 4: The ResNet20-like model used in our experiments. We build residual connections from the first convolution layer to the last layer in the same convolution block.

Supplementary Material

A An Attempt to Combine Purin and Residual Connection

In this section, we provide an idea that introduces the Purin mechanism to a model with residual connection. The aim of this experiment is to examine whether Purin can work with residual connections, rather than to present that the model with the Purin mechanism can outperform the vanilla ResNet20 model (He et al. 2016). In this section, we used a ResNet20-like model (shown in Fig. 4) and evaluated it on the CIFAR-10 dataset (Krizhevsky and Hinton 2009). We did not use data augmentation methods in our experiments.

A.1 Residual implementation and results

In the proposed Purin residual model, we use the same parameter configuration as the vanilla ResNet20 model without the batch normalization layers. The residual connections (1×1 convolution layers with Purin mechanism) in the proposed model will sum up the results of $f(\cdot)$ in the beginning layer and the end layer. Eqs. (16) and (17) present how to calculate the final output Y for the residual end layer, X_{res} is the residual input from the beginning layer. Experimental results over five independent experiments are shown in Table 9.

$$Y = g_{stp} \odot W_{out} \odot (f_{end}(W_{in}X) + X_{res}) \quad (16)$$

$$X_{res} = g_{stp}^{residual} \odot W_{out}^{residual} \odot f(W_{in}^{residual} X^{begin}) \quad (17)$$

Model	Accuracy
vanilla ResNet20	69.53 ± 0.61
Ours	69.35 ± 2.64

Table 9: Accuracy (%) on the CIFAR-10 dataset. The average values and standard deviations are given.

Compared to the vanilla ResNet20 architecture, the proposed Purin residual model achieves a comparable classification accuracy with a slight drop of 0.18%. This result indicates that the proposed Purin mechanism can be incorporated into residual connections without causing severe degradation in classification accuracy.

B Short-term Factor Behavior Visualization

In this section, we visualize the short-term factor g_{stp} curve in a trained Purin+VGG11 model in an evaluation batch on the CIFAR-100 dataset (Krizhevsky and Hinton 2009). We choose three neurons from each convolution layer or fully connected layer as examples. We number the layers in VGG11 architecture from 0 to 16 (including all maxpooling layers and a flatten layer): Convolution_0, Maxpooling_1, Convolution_2, etc. The g_{stp} curves for each layer are shown in Fig 5.

As shown in Fig. 5, the values of g_{stp} in different layers do not saturate at either the upper or lower boundary within a batch. Therefore, the short-term factor provides mild scaling to the output of a neuron. The increasing trend of g_{stp} can be attributed to the leaky ReLU function used in this paper, which scales the negative outputs by 0.01 and thus weakens their influence on g_{stp} updates.

C Additional Trainable Parameters Overhead Analysis

In this section, we report the additional trainable parameter overhead of architectures with the Purin mechanism. Suppose a convolutional layer has C_{in} input channels, C_{out} output channels, and a convolution kernel of size $k \times k$. The number of trainable parameters in this layer is $C_{out}C_{in}k^2$. For fully connected layers, the number of trainable parameters is $N_{in} \times N_{out}$, where N_{in} and N_{out} are the number of input neurons and output neurons, respectively. In Purin, we add extra trainable weight W_{out} to the convolutional layers and the fully connected layers, which adds C_{out} additional parameters to the convolutional layers and N_{out} additional parameters to the fully connected layers. Therefore, the overhead of additional trainable parameters for each convolutional layer and fully connected layer in architectures with the Purin mechanism is calculated by Eq (18) and Eq (19), respectively.

$$\frac{C_{out}}{C_{out}C_{in}k^2} = \frac{1}{C_{in}k^2} \quad (18)$$

$$\frac{N_{out}}{N_{in}N_{out}} = \frac{1}{N_{in}} \quad (19)$$

The additional trainable parameter overheads for all three matched models in the main paper, AlexNet, VGG11, and

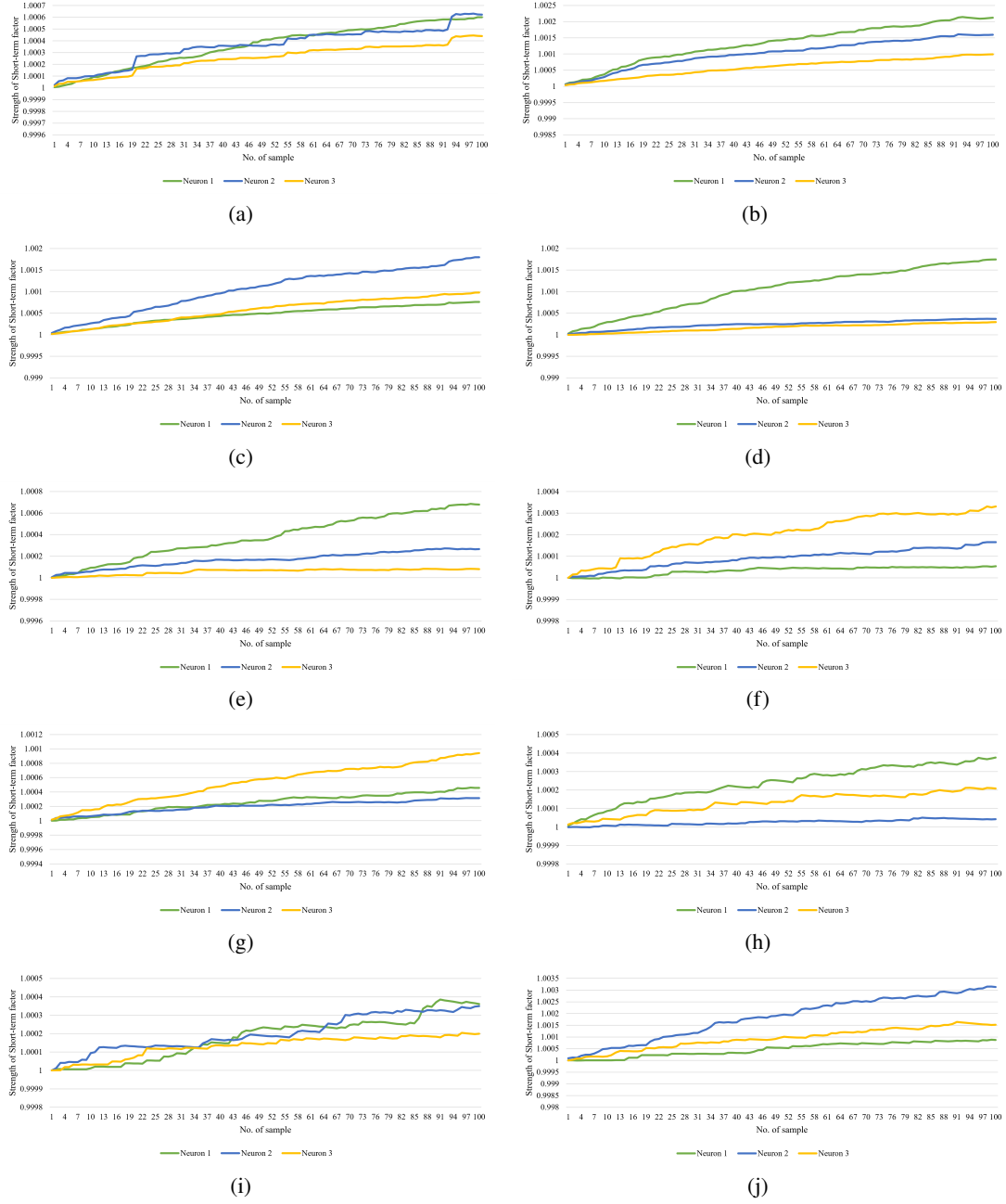


Figure 5: g_{stp} curves in an evaluation batch in convolution layers and fully connected layers in the Purin+VGG11 model. The last fully connected layer does not use g_{stp} . (a) Convolution_0. (b) Convolution_2. (c) Convolution_4. (d) Convolution_5. (e) Convolution_7. (f) Convolution_8. (g) Convolution_10. (h) Convolution_11. (i) fully connected layer_14. (j) fully connected layer_15.

GoogLeNet, are presented in Table 10. We assume that the size of the input image for all models is 224×224 and the number of categories is 100. Since the last fully connected layer in all three models disables W_{out} and g_{stp} , it does not contribute to the number of additional parameters in the models.

Model	Extra trainable parameter	Overhead
AlexNet	9344	0.0162%
VGG11	10944	0.0084%
GoogLeNet	7280	0.1199%

Table 10: Relative overhead of additional trainable parameters for a dataset with 224×224 images, 100 categories.

The additional trainable parameter overhead of Purin is below 0.2% for all three matched architectures. Therefore, the accuracy improvements in models with the Purin mechanism are unlikely to be explained simply by a large increase in the trainable model capacity.

D Random Seed used in this Paper

To split the validation sets from the training sets on the CIFAR-100 and 102 Category Flower datasets, we generated random numbers with a manually set random seed 42. The same random seed was also used to split the training, validation, and testing sets on the UCM and AID datasets. This random seed was also used in experiments in the supplementary material.

References

- Abbott, L. F.; Varela, J.; Sen, K.; and Nelson, S. 1997. Synaptic depression and cortical gain control. *Science*, 275(5297): 221–224.
- Bellec, G.; Salaj, D.; Subramoney, A.; Legenstein, R.; and Maass, W. 2018. Long short-term memory and learning-to-learn in networks of spiking neurons. *Advances in neural information processing systems*, 31.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- Hennig, M. H. 2013. Theoretical models of synaptic short term plasticity. *Frontiers in computational neuroscience*, 7: 45.
- Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.
- Hou, Q.; Zhou, D.; and Feng, J. 2021. Coordinate attention for efficient mobile network design. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 13713–13722.
- Hu, J.; Shen, L.; and Sun, G. 2018. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 7132–7141.
- Hu, Y.; Zheng, Q.; Li, G.; Tang, H.; and Pan, G. 2024. Toward large-scale spiking neural networks: A comprehensive survey and future directions. *arXiv preprint arXiv:2409.02111*.
- Kaur, R.; and Singh, S. 2023. A comprehensive review of object detection with deep learning. *Digital Signal Processing*, 132: 103812.
- Krizhevsky, A.; and Hinton, G. 2009. Learning multiple layers of features from tiny images. *Handbook of Systemic Autoimmune Diseases*, 1(4).
- Krizhevsky, A.; Sutskever, I.; and Hinton, G. E. 2012. ImageNet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.
- Liu, H.; Dai, Z.; So, D.; and Le, Q. V. 2021. Pay attention to mlps. *Advances in neural information processing systems*, 34: 9204–9215.
- Maguire, J. F.; and Howe, P. D. 2016. Failure to detect meaning in RSVP at 27 ms per picture. *Attention, Perception, & Psychophysics*, 78(5): 1405–1413.
- Marblestone, A. H.; Wayne, G.; and Kording, K. P. 2016a. Toward an integration of deep learning and neuroscience. *Frontiers in computational neuroscience*, 10: 215943.
- Marblestone, A. H.; Wayne, G.; and Kording, K. P. 2016b. Toward an Integration of Deep Learning and Neuroscience. *Frontiers in Computational Neuroscience*, Volume 10 - 2016.
- Markram, H.; Gerstner, W.; and Sjöström, P. J. 2011. A history of spike-timing-dependent plasticity. *Frontiers in synaptic neuroscience*, 3: 4.
- Mohsenzadeh, Y.; Qin, S.; Cichy, R. M.; and Pantazis, D. 2018. Ultra-Rapid serial visual presentation reveals dynamics of feedforward and feedback processes in the ventral visual pathway. *Elife*, 7: e36329.
- Nilsback, M.-E.; and Zisserman, A. 2008. Automated Flower Classification over a Large Number of Classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*.
- Regehr, W. G. 2012. Short-term presynaptic plasticity. *Cold Spring Harbor perspectives in biology*, 4(7): a005702.
- Simonyan, K.; and Zisserman, A. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; and Salakhutdinov, R. 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958.
- Szegedy, C.; Liu, W.; Jia, Y.; Sermanet, P.; Reed, S.; Anguelov, D.; Erhan, D.; Vanhoucke, V.; and Rabinovich, A. 2015. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1–9.
- Taherkhani, A.; Belatreche, A.; Li, Y.; Cosma, G.; Maguire, L. P.; and McGinnity, T. M. 2020. A review of learning in biologically plausible spiking neural networks. *Neural Networks*, 122: 253–272.
- Tavanaei, A.; Ghodrati, M.; Kheradpisheh, S. R.; Masquelier, T.; and Maida, A. 2019. Deep learning in spiking neural networks. *Neural networks*, 111: 47–63.

- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L. u.; and Polosukhin, I. 2017. Attention is All you Need. In Guyon, I.; Luxburg, U. V.; Bengio, S.; Wallach, H.; Fergus, R.; Vishwanathan, S.; and Garnett, R., eds., *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Wang, E. K.; Chen, C.-M.; Hassan, M. M.; and Almogren, A. 2020. A deep learning based medical image segmentation technique in Internet-of-Medical-Things domain. *Future Generation Computer Systems*, 108: 135–144.
- Whittington, J. C.; and Bogacz, R. 2019. Theories of error back-propagation in the brain. *Trends in cognitive sciences*, 23(3): 235–250.
- Xia, G.-S.; Hu, J.; Hu, F.; Shi, B.; Bai, X.; Zhong, Y.; Zhang, L.; and Lu, X. 2017. AID: A Benchmark Data Set for Performance Evaluation of Aerial Scene Classification. *IEEE Transactions on Geoscience and Remote Sensing*, 55(7): 3965–3981.
- Xu, Q.; Li, Y.; Shen, J.; Zhang, P.; Liu, J. K.; Tang, H.; and Pan, G. 2022. Hierarchical spiking-based model for efficient image classification with enhanced feature extraction and encoding. *IEEE Transactions on Neural Networks and Learning Systems*, 35(7): 9277–9285.
- Yang, Y.; and Newsam, S. 2010. Bag-Of-Visual-Words and Spatial Extensions for Land-Use Classification. In *ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (ACM GIS)*.
- Zenke, F.; and Ganguli, S. 2018. Superspike: Supervised learning in multilayer spiking neural networks. *Neural computation*, 30(6): 1514–1541.